Methodology Report: Automated Financial Document Extraction using Local LLMs

Course: CAP 5619 - Artificial Intelligence for FinTech

Assignment: Assignment 2 - LLM Document Analysis

Model Used: `deepseek-r1:8b` (via Local Ollama)

GitHub: github.com/proyear/CAP5619.git

1. Objective and Approach

The objective of this assignment was to build an automated data pipeline capable of downloading unstructured financial documents (Form 8-K) from the SEC EDGAR database and utilizing a Large Language Model (LLM) to perform Named Entity Recognition (NER). Specifically, the pipeline was tasked with extracting new product announcements and descriptions (under 180 characters) into a structured, pipe-delimited format, eventually scaling to process over 100 companies.

To ensure data privacy and avoid API costs—a key consideration in modern FinTech environments—the extraction was performed using a local reasoning model (`deepseek-r1:8b`) hosted via Ollama, rather than relying on commercial endpoints like OpenAI.

2. Python Features and Methodology

The pipeline was developed in a Jupyter Notebook using several core Python libraries and concepts discussed in class:

- **RESTful APIs & `requests`:** Initiated the pipeline by querying the SEC's `company_tickers.json` endpoint to dynamically fetch the Central Index Keys (CIK) for top S&P 500 companies. Passed custom HTTP headers (`User-Agent`) to comply with SEC automated traffic guidelines.
- **Data Wrangling with `pandas`:** The JSON response was parsed into a Pandas DataFrame, allowing easy filtering, isolation, and iteration through our target list of 150 companies.
- **Web Scraping & Parsing (`bs4`):** Used `BeautifulSoup` to parse the XML/Atom feeds from the SEC EDGAR search endpoints to locate the latest 8-K filings. It was suggested to check additional filings if needed, but held off as I'm not sure that was a requirement.
- **Regular Expressions (`re`):** While the LLM handled the heavy lifting of NER, traditional Regular Expressions were used to clean the LLM's output, specifically stripping out `<think>` tags generated by the DeepSeek reasoning model before saving the data.

3. Engineering Challenges and Resolutions

During the development of the pipeline, several challenges were encountered, real-world data retrieval and formatting issues.

Challenge 1: Missing XML Parsers

- **Issue:** The initial attempt to parse the SEC Atom feed failed with a `FeatureNotFound` error because the Python environment lacked the `lxml` dependency.
- **Resolution:** Rather than forcing environment installations, the code was adapted to use Python's built-in `'html.parser'`. Because the SEC feed uses standard top-level tags, the native parser successfully navigated the document tree without external dependencies.

Challenge 2: The JavaScript/Index Page Block

- **Issue:** Our initial text extraction returned a generic SEC warning ("This page uses Javascript") rather than the 8-K text. The Atom feed was pointing to the filing's HTML Index Page rather than the raw document.
- **Resolution:** A programmatic string manipulation trick was applied. By identifying the index URL and using Python's `.replace('-index.htm', '.txt')`, the navigation boilerplate was bypassed and successfully downloaded the raw, unstructured legal text submission.

Challenge 3: Server Timeouts and Rate Limiting

- **Issue:** When scaling the loop to process 150 companies, the pipeline crashed around the 73rd document with a `ReadTimeout`. It's assumed the SEC servers dropped the connection due to traffic throttling and/or it was late in the evening and servers may have been taken offline.
- **Resolution:** Engineered a resilient retry loop and ran the code in the morning. Wrapping the `requests.get()` call in a `try-except` block, introduced an explicit `timeout=15` parameter, increased the maximum retries to 5, and raised the `time.sleep()` from 0.2 seconds to 1 full second. This seemed to respect the SEC's infrastructure and allowed the loop to complete successfully.

4. Findings and Output Analysis

The final pipeline successfully processed 73 documents after a second iteration that consisted of changes to the script to filter out announcements that were not about New Products. The data was exported to `extracted_8k_products.csv`.

Analysis of the CSV Data: A review of the CSV file confirmed that the local `deepseek-r1:8b` model successfully adhered to the structural constraints: exactly five pipe-delimited fields were returned per document, and the product descriptions were kept brief.

Notably, many of the filings resulted in an empty `new_product` field. Upon review, this indicates a highly successful extraction. SEC Form 8-K filings are utilized for material corporate events (e.g., executive changes, earnings reports, acquisitions). Most of these documents do not contain product announcements. Instead of "hallucinating" false products to fill the fields, the local LLM correctly identified the absence of product announcements in the unstructured text and left the fields blank, proving the model's reliability in processing strict financial data.